

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

PoolManager

0x0000000000004444c5dc75cB358380D2e3dE08A90

Ethereum

Compiler: v0.8.26+commit.8a97fa7a

Date: October 7, 2026

OVERALL RISK RATING *AI-assisted audit. Not a substitute for manual review.*

Low

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. This is the canonical Uniswap v4 PoolManager singleton contract and its supporting libraries. The core accounting model (transient-storage delta tracking, unlock/lock reentrancy guard, flash-accounting via take/settle/mint/burn) is sound and consistent with the documented design, and no critical or high-severity fund-draining vulnerabilities were identified. Findings are limited to style/documentation issues, floating pragmas in library/interface files, a minor settlement incompatibility with fee-on-transfer/rebasing tokens, and low-impact centralization risk around the protocol fee controller.

Findings (5)

F-001

Low

Settlement logic is incompatible with fee-on-transfer / rebasing / deflationary tokens

src/PoolManager.sol:247–262

DESCRIPTION

`\_settle` computes `paid = reservesNow - reservesBefore` using the post-transfer balanceOf. For a fee-on-transfer or negative-rebasing token, the amount received can be less than the amount sent, and for a positive-rebasing token held by the manager between sync/settle calls the balance could unexpectedly change for reasons unrelated to the caller's transfer. Because Solidity 0.8 reverts on underflow, any currency balance decrease between `sync` and `settle` that is not attributable to the caller's deposit (or a negative-fee token) will cause the whole unlock() transaction to revert.

IMPACT

Pools created with fee-on-transfer, rebasing, or otherwise balance-mutating ERC-20 tokens may be unusable: swaps/liquidity operations relying on `settle()` will revert, resulting in denial of service for interactions with such tokens. This does not lead to fund loss but can make certain legitimate token pools non-functional.

RECOMMENDATION

Document explicitly (and enforce off-chain/at the periphery level) that fee-on-transfer, rebasing, and other balance-mutating tokens are not supported by PoolManager's accounting model. Consider guarding against underflow with a clearer custom revert (e.g. `AmountNotSettled`) instead of relying on the implicit Solidity panic, to improve integrator debuggability.

```
uint256 reservesBefore = CurrencyReserves.getSyncedReserves();
uint256 reservesNow = currency.balanceOfSelf();
paid = reservesNow - reservesBefore; // reverts (panic) if negative for non-standard tokens
CurrencyReserves.resetCurrency();
```

F-002 · SWC-103

Low

Floating pragma used across library and interface files

src/libraries/Hooks.sol:2–2

DESCRIPTION

While `PoolManager.sol` itself locks the compiler version to `0.8.26`, nearly all supporting libraries and interfaces (`Hooks.sol`, `Pool.sol`, `Position.sol`, `LPFeeLibrary.sol`, `Currency.sol`, `PoolKey.sol`, `TickMath.sol`, `SafeCast.sol`, `NoDelegateCall.sol`, `IHooks.sol`, `IPoolManager.sol`, `ProtocolFees.sol`, `ERC6909.sol`, `CurrencyDelta.sol`, `NonzeroDeltaCount.sol`, `CurrencyReserves.sol`, `Lock.sol`, `Extload.sol`, `Exttload.sol`, `CustomRevert.sol`, `TickBitmap.sol`, `SqrtPriceMath.sol`, `SwapMath.sol`, etc.) declare a floating pragma (`0.8.0` or `0.8.24`). If these libraries were ever compiled/deployed independently or reused by third parties with a different compiler version than the one actually used for the deployed bytecode, subtle semantic differences between compiler versions could be introduced.

IMPACT

Low, for the specific deployed bytecode this is not exploitable since the whole project is compiled together with a single compiler version (0.8.26), but it is a best-practice violation that creates risk for downstream integrators who import these libraries directly.

RECOMMENDATION

Pin all library/interface files to the exact compiler version used for the production build (`pragma solidity 0.8.26;`) to guarantee deterministic compilation across all consumers of the codebase.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

library Hooks {
  ...
}
```

F-003

Low

Centralization risk: owner can arbitrarily (re)assign the protocol fee controller

src/ProtocolFees.sol:25–28

DESCRIPTION

`setProtocolFeeController` is gated only by `onlyOwner` with no timelock, multisig enforcement, or event-based delay. The appointed `protocolFeeController` can subsequently call `setProtocolFee` (capped at `MAX_PROTOCOL_FEE` = 0.1%) and `collectProtocolFees` to withdraw all currently accrued protocol fees to an arbitrary recipient.

IMPACT

If the `owner` key is compromised or acts maliciously, it can immediately redirect the protocol-fee-controller role to an attacker-controlled address, which can then set the maximum allowed protocol fee on any pool and sweep all previously/subsequently accrued protocol fees. Impact is bounded to protocol fees only (not LP principal or swap proceeds), capped at 0.1% per swap.

RECOMMENDATION

Consider adding a timelock or multi-sig/governance delay on `setProtocolFeeController`, and emit sufficient events (already emitted) so monitoring systems can alert quickly on changes. Document the trust assumptions clearly for integrators/LPs.

```
function setProtocolFeeController(address controller) external onlyOwner {
    protocolFeeController = controller;
    emit ProtocolFeeControllerUpdated(controller);
}
```

F-004

Informational

NatSpec/documentation mismatch on collectProtocolFees lock-state requirement

src/interfaces/IProtocolFees.sol:34–41

DESCRIPTION

The interface NatSpec states ``@dev This will revert if the contract is unlocked``, implying ``collectProtocolFees`` should only be callable while the PoolManager is locked. However, the implementation in ``ProtocolFees.sol`` contains no check against ``Lock.isUnlocked()`/`_isUnlocked()``; the only guard is ``msg.sender == protocolFeeController`` and that the requested currency isn't currently synced.

IMPACT

No direct security impact since the function remains properly access-controlled, but the misleading documentation could cause integrators/auditors to make incorrect assumptions about when the function is safe to call (e.g., assuming it cannot be invoked mid-unlock-callback).

RECOMMENDATION

Update the NatSpec comment to accurately reflect the actual guard (``InvalidCaller`` + ``ProtocolFeeCurrencySynced`` checks only), or add an explicit ``lockdown`/unlocked-state` check if that was the original intended invariant.

```
/// @dev This will revert if the contract is unlocked
function collectProtocolFees(address recipient, Currency currency, uint256 amount)
external
returns (uint256 amountCollected);
```

F-005

Informational

Hook contracts are fully trusted external call targets chosen by pool creators

src/libraries/Hooks.sol:136–151

DESCRIPTION

Every pool-mutating action (``initialize``, ``modifyLiquidity``, ``swap``, ``donate``) performs a low-level ``call`` into the pool's ``hooks`` address, which is an arbitrary address chosen by whoever calls ``initialize`` for that pool (subject only to the address-flag validity check in ``isValidHookAddress``). A malicious or buggy hook can revert, consume excess gas, or (if it has the appropriate return-delta permission flags encoded in its address) manipulate the swap/liquidity deltas within the bounds enforced by ``HookDeltaExceedsSwapAmount``.

IMPACT

This is an inherent, documented part of the Uniswap v4 architecture (hooks are opt-in per pool), so it does not constitute a vulnerability in PoolManager itself, but integrators and LPs interacting with a given pool must independently trust/audit that pool's specific hook contract, since PoolManager provides no on-chain hook allow-listing or reputation mechanism.

RECOMMENDATION

No code change needed in PoolManager; ensure front-ends/integrators surface the pool's hook address prominently and encourage LPs/swappers to verify hook contracts before interacting, since hooks are a distinct trust boundary from the core protocol.

```
function callHook(IHooks self, bytes memory data) internal returns (bytes memory result)
{
    bool success;
    assembly ("memory-safe") {
        success := call(gas(), self, 0, add(data, 0x20), mload(data), 0, 0)
    }
    ...
}
```


Gas Optimizations

Cache `CurrencyReserves.getSyncedCurrency()` result in `_settle`

src/PoolManager.sol:247–262

`_currency` is already read once at the top of `_settle`; ensure subsequent branches reuse the local variable instead of re-reading transient storage (already mostly done), no functional change needed, but double-check call sites in future refactors avoid redundant `tload`s.

Use unchecked arithmetic where safe additions are already proven non-overflowing

src/ProtocolFees.sol:55–59

`_updateProtocolFees` already wraps the increment in `unchecked`; confirm similar safe increments elsewhere (e.g., repeated `+=` on `protocolFeesAccrued`) are consistently wrapped to save redundant overflow checks, since protocol fee amounts are bounded by token supply constraints documented in Pool.sol.

Mark `sync` as not needing SLOAD-heavy branch for native currency

src/PoolManager.sol:216–224

Consider short-circuiting `currency.isAddressZero()` checks with a single cached boolean if `sync` is called frequently in hot paths, reducing repeated branch evaluation overhead for ERC20-heavy workloads (minor, call-frequency dependent).

Appendix - Metadata

Contract	PoolManager
Address	0x0000000000004444c5dc75cB358380D2e3dE08A90
Chain	Ethereum
Compiler	v0.8.26+commit.8a97fa7a
Audit ID	portfolio-uniswap-v4-poolmanager
Date	10/7/2026

Files analyzed

```
[user] src/PoolManager.sol
[user] src/libraries/Hooks.sol
[user] src/libraries/Pool.sol
[user] src/libraries/SafeCast.sol
[user] src/libraries/Position.sol
[user] src/libraries/LPFeeLibrary.sol
[user] src/types/Currency.sol
[user] src/types/PoolKey.sol
[user] src/libraries/TickMath.sol
[user] src/NoDelegateCall.sol
[user] src/interfaces/IHooks.sol
[user] src/interfaces/IPoolManager.sol
[user] src/interfaces/callback/IUnlockCallback.sol
[user] src/ProtocolFees.sol
[user] src/ERC6909Claims.sol
[user] src/types/PoolId.sol
[user] src/types/BalanceDelta.sol
[user] src/types/BeforeSwapDelta.sol
[user] src/libraries/Lock.sol
[user] src/libraries/CurrencyDelta.sol
[user] src/libraries/NonzeroDeltaCount.sol
[user] src/libraries/CurrencyReserves.sol
[user] src/Extload.sol
[user] src/Exttload.sol
[user] src/libraries/CustomRevert.sol
[user] src/libraries/ParseBytes.sol
[user] src/libraries/TickBitmap.sol
[user] src/libraries/UnsafeMath.sol
[user] src/libraries/FixedPoint128.sol
[user] src/libraries/SqrtPriceMath.sol
[user] src/libraries/SwapMath.sol
[user] src/types/Slot0.sol
[user] src/libraries/ProtocolFeeLibrary.sol
[user] src/libraries/LiquidityMath.sol
[user] src/libraries/FullMath.sol
[user] src/interfaces/external/IERC20Minimal.sol
[user] src/libraries/BitMath.sol
```

```
[user] src/interfaces/external/IERC6909Claims.sol
[user] src/interfaces/IProtocolFees.sol
[user] src/interfaces/IExtsload.sol
[user] src/interfaces/IExttload.sol
[dependency] lib/solmate/src/auth/Owned.sol
[user] src/ERC6909.sol
[user] src/libraries/FixedPoint96.sol
```


MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-uniswap-v4-poolmanager)

<https://cryptohawking.com/audit/manual?ref=portfolio-uniswap-v4-poolmanager>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>